Scribble to Digital

Team ID :SWTID-2026-1685

Team Size : 4

Team Leader **:** JEYASURYA S

Team Member : AFROSE BANU S

Team Member : SENTHURDIVYA A

Team Member : ASWINTH T

Project Description :

Handwritten notes are widely used by students, teachers, and professionals to quickly record ideas, reminders, and important information. However, these notes can sometimes be difficult to read, organize, or store digitally, especially when the handwriting is unclear or the lighting conditions are poor. Converting such handwritten content into digital format often requires manual typing, which can be time-consuming and inefficient.

The **Scribble to Digital** system is designed to solve this problem by automatically transforming handwritten notes into well-structured digital text. Users can upload an image of their handwritten notes, and the system processes the image to improve its clarity. It then uses **Optical Character Recognition (OCR)** to identify and extract the text from the image.

After extracting the text, the system applies **Generative AI** to refine the content by correcting grammar, improving sentence structure, and organizing the information into readable paragraphs. Additionally, the system analyzes the text to identify important action items and converts them into a clear **To-Do task list**.

The application is built using **Streamlit** to create an interactive user interface, while **Python** is used to handle image processing, text extraction, and AI integration. This solution helps users quickly convert handwritten notes into organized digital content, making it easier to store, edit, and share information.

Use Case Scenarios :

Scenario 1: Uploading and Processing Handwritten Notes

In this scenario, a user uploads an image of handwritten notes through the application interface. The system first processes the image to improve its quality. Image enhancement techniques are

applied to make the text clearer and easier to read. These steps help remove noise, adjust brightness, and improve contrast so that the handwritten content becomes more visible.

Once the image quality is improved, the system sends it to the OCR module to detect and extract the written characters. This step helps convert the handwritten information into machine-readable text.

Output:

- Enhanced image with improved clarity
- Extracted handwritten text in digital form

Scenario 2: Text Refinement and Formatting

The text extracted from OCR may contain errors such as incorrect words, missing letters, or unclear sentence structures. In this scenario, the extracted text is processed by a Generative AI model to improve its readability.

The AI analyzes the context of the text and performs several improvements, such as correcting spelling mistakes, improving grammar, and restructuring sentences. It also organizes the text into properly formatted paragraphs so that the content becomes easier to read and understand.

Output:

- Corrected and refined digital text
- Well-structured paragraphs with improved grammar and clarity

Scenario 3 : Identification of Actionable Tasks

Handwritten notes often contain reminders, action items, or tasks mixed with general information. In this scenario, the AI system analyzes the processed text to identify sentences that indicate tasks or actions.

The system detects keywords and sentence patterns that represent tasks and separates them from normal text. These tasks are then organized into a clear list, helping users quickly identify important actions that need to be completed.

Output:

- A structured list of extracted tasks
- Clear separation between general notes and actionable items

Scenario 4: Digital Storage and Sharing .

After processing and organizing the content, the final digital notes can be stored in the system for future reference. Users can also download or share the notes in different digital formats. This allows easy access to the information from any device and simplifies collaboration with others.

Output:

- Downloadable digital notes

- Shareable document for easy access and collaboration

Architecture Overview :

Scribble to Digital is a smart web-based application that helps convert handwritten notes into organized digital text. The system is designed to simplify the process of transforming handwritten information into readable and structured digital content. It provides an easy-to-use interface where users can upload images of handwritten notes and receive a clean digital version of the text.

The application is developed using **Stream lit**, which provides an interactive and user-friendly interface for uploading images, processing data, and displaying results. The backend processing is handled using **Python**, which coordinates the different modules involved in the system.

To improve the quality of uploaded images, the system uses **OpenCV** for image preprocessing tasks such as noise removal, contrast enhancement, and grayscale conversion. These steps help increase the accuracy of text recognition.

The system then uses an **OCR (Optical Character Recognition) engine**, such as Tesseract or Easy OCR, to detect and extract text from the enhanced image. The extracted text is further processed using **Generative AI with the Gemini model**, which improves the text by correcting grammar, restructuring sentences, and organizing the content into readable notes.

Finally, the system presents the processed output to the user in a structured format, which may include cleaned digital text and an optional list of extracted tasks or action items. This architecture ensures efficient processing of handwritten notes and improves the overall usability of the application.

Core Technologies :

- **Streamlit**: Used to build the web interface of the application. It allows users to upload images, start processing, and view the generated results in an interactive environment.
- **Generative AI (Gemini Model)**: Used to improve the extracted text by correcting grammar, refining sentence structure, and identifying important tasks from the notes.
- **OpenCV Library**: Helps in image preprocessing by improving image quality through operations like grayscale conversion, noise reduction, and contrast enhancement.
- **OCR Technology (Tesseract or Easy OCR)**: Responsible for reading and extracting text from the processed handwritten images and converting it into machine-readable format.
- **Python Programming Language**: Acts as the main backend language that connects all components such as image processing, OCR, AI integration, and the user interface.
- **Document Export Libraries**: Tools such as PDF and Word document libraries are used to allow users to download the final digital notes in different file formats.

Component-Wise Architecture :

Component	Description
Streamlit UI	Provides a simple interface forusersto upload handwritten images and view the results.
Image Processing (OpenCV)	Enhances the image by converting to grayscale, improving contrast, and removing noise.
OCR Module	Extracts text from the processed handwritten image and converts it into digital text.
AI Processing	Corrects grammar, improves sentence structure, and organizes the text. It can also detect tasks from the notes.

Pre-requisites :

1. **Google AI Studio Account** – Required to create and use the Gemini API key.
2. **Python Installed** – Python version 3.8 or higher should be installed on thesystem.

1. **Streamlit Framework**– Interface Needed to build and run the web application

4. **OCR Tool Setup** – Install an OCR engine like Tesseract or EasyOCR for text recognition.

5. **Required Python Libraries** – Install necessary libraries such as OpenCV, Pillow, and AI integration libraries.

6.**Export Libraries** – Tools like ReportLab and python-docx are required to generate PDF or Word files for downloading the final notes.

Project Workflow:

1. AI Model Setup and Configuration Step 1.1: Create Gemini API Access

- Sign in or create an account in Google AI Studio.
- Generate an API key to access the Gemini model.
- Activate the Gemini model required for the project.
- Test the API connection using a simple prompt to confirm it works properly.

Step 1.2: Environment Setup

- Save the API key securely in a .env configuration file.

- Use the **python.env** libraryto load the environment variables in the project.

- Configure model settings such as response length, creativity level, and output format.

- Verify that the API connects correctly with the Streamlit application.

Step 1.3: Prompt Creation for AI Processing Design prompts that help the AI improve extracted text.

Prompts are used for:

- Correcting grammar and spellingo Improving sentence structure
- Converting messy text into organized paragraphs
- Create additional prompts to detect and extract tasks from the notes.

2. System Development

Step 2.1: Build Streamlit Application Layout

- Organize the project into separate modules for better management.
- Design the main Streamlit interface including:
- Application title and description
- Image upload section
- Processing buttons
- Output display area.

Step 2.2: Image Processing Implementation

- Allow users to upload handwritten images in formats such as JPG or PNG.
- Use **OpenCV** to improve image quality by: o Converting images to grayscale o Adjusting brightness and contrast o Removing unwanted noise

o Applying threshold filters

- Display the processed image for user preview.

Step 2.3: Text Extraction Using OCR

- Integrate an OCR tool like **Tesseract**or **EasyOCR**.
- Extract handwritten text from the processed image.
- Clean the extracted text by removing unwanted characters or symbols.
- Show the raw OCR output for comparison.

Step 2.4: AI-Based Text Enhancement

- Send the extracted text to the Gemini AImodel.
- The AI performs:
  - Grammar correctiono Sentence improvement
  - Context-based word completion
  - Formatting into readable notes
- The system also identifies action items and creates a separate task list.

<u>3. User Interface Development</u>

Step 3.1: Interactive Components

- Implement interface elements such as:

st.file_uploader() for uploading images

st.image() for previewing images

st.button() for triggering processing steps

st.spinner() for loading indicators

st.text_area() for displaying extracted text.

Step 3.2: Result Display

The application displaysresults in different sections:

- Enhanced Image
- Raw OCR Extracted Text
- AI Corrected Digital Notes
- Automatically Generated To-Do Tasks
- Optional features may include hiding the raw OCR output and enabling downloads.

Step 3.3: Export and Download Options

- Allow users to export results in different formats.
- Generate downloadable files such as:
  - PDF documents
  - Word documents
- Provide download buttons for saving the processed notes.

4. Testing and Improvement

Step 4.1: System Testing

- Test the system using different handwriting styles such as:
  - Clear handwritingo Poor or messy handwriting o Notes containing both text and tasks
  - Bullet-point lists.
- Verify the accuracy of both OCRextraction and AI processing.

Step 4.2: AI Output Evaluation

- Check the AI-generated contentfor:
  - Correct grammaro Logical sentence flow o Proper formatting
  - Accurate identification of tasks.

Step 4.3: Performance Optimization

- Improve image preprocessing efficiency.
- Reduce unnecessary API usage to save processing time.
- Implement error handling for cases such as image upload errors, OCR failure, or API issues.

Step 4.4: Export Verification

- Confirm that generated PDF and Word files are correctly formatted.
- Ensure files open correctly on different devices.
- Test downloads across multiple web browsers.

Milestone 1: AI Setup & Initialization

Objective:

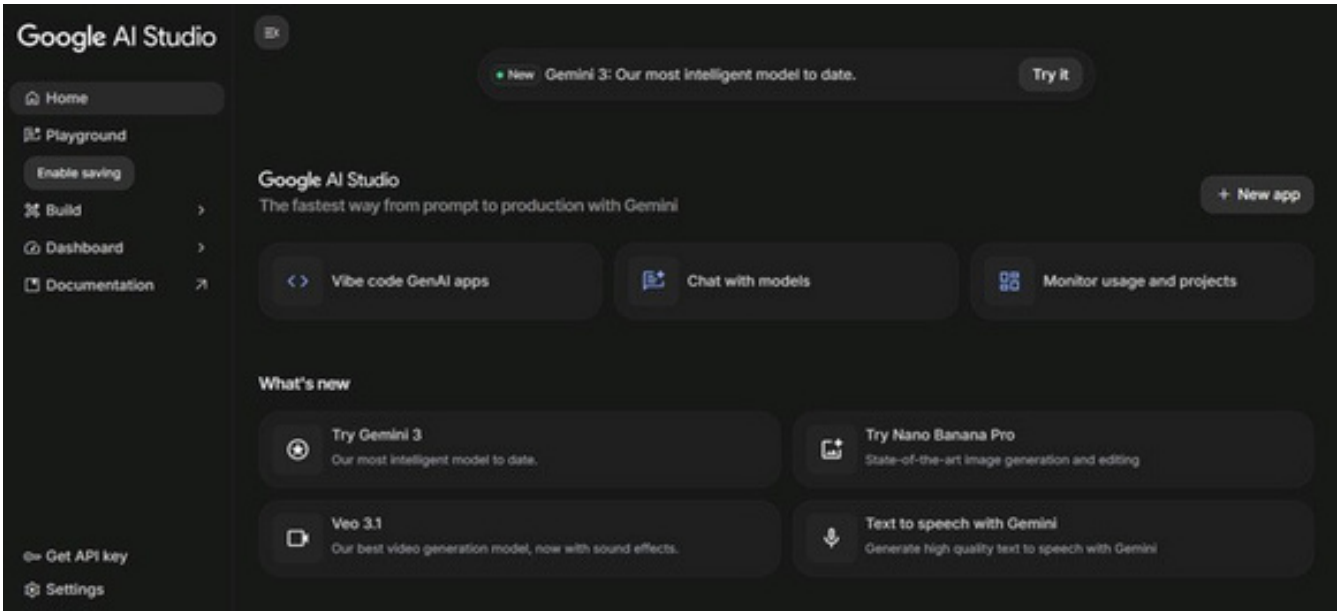
Establish the AI backbone required to transform handwritten content into structured digital text using **Google Gemini 2.5 Flash**.

The system will:

- Refine messy OCR output
- Fix grammatical and spelling errors
- Predict missing words using context
- Identify actionable tasks
- Convert handwritten notes into structured digital documents.

Activity 1.1: Gemini API Configuration

Step 1: Access Google AI Studio



- Open: https://makersuite.google.com/app/apikey
- Sign in with your Google account
- Accept usage policies

Step 2: Generate API Key

- Click **"CreateAPI Key"**
- Copy and store securely
- Save inside .env file

```
scribble_to_digital/
    app.py
    .env
```

Add:

```
GEMINI_API_KEY=your_api_key_here
```

Step 3: Choose AI Model Selected Model: **Gemini2.5Flash**

Why this model?

- Low latency
- Cost-efficient
- Optimized for text processing
- Ideal for rewriting and structured extraction

Activity 1.2: Environment Setup

Required Libraries

Install dependencies:

```
pip install streamlit google-generativeai python-dotenv pillow opencv-python pytesseract
```

<u>Purpose of Libraries</u>

Library	Function
Stream lit google- Generative ai	Web interface- Connects to Gemini
python-dot env	Loads API keys
OpenCV	Image enhancement
Pytesseract	Extract text
Pillow	Image handling

Activity 1.3: API Testing

Create a simple test inside app.py:

```python
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv

# Load API key
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# Select model
model = genai.GenerativeModel("models/gemini-2.5-flash")

st.title("Gemini API Test - Scribble to Digital")

if st.button("Test AI Connection"):
    response = model.generate_content(
        "Correct this sentence: i am go school tomorrow"
    )
    st.write("AI Response:")
    st.write(response.text)
```

Test 1 – Sentence Correction

Input:

"tomorrow meting at 5 pls bring documnts"

Expected Result:

"Tomorrow's meeting is at 5 PM. Please bring the documents."

Test 2 – Task Extraction

Input:

"Finish assignment submit by friday call rahul" Expected Result:

- Clean paragraph
- Extracted tasks:

  o Finish assignment

  o Submit by Friday

o Call Rahul

Milestone 2: Application Development (Streamlit)

Objective:

Develop the complete working interface integrating OCR and AI.

Activity 2.1: Application Initialization

Create main file: app.py

```
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv
from PIL import Image
import pytesseract
import cv2
import numpy as np
```

- Configure model

```
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")
```

Activity 2.2: OCR Processing Module

```python
import cv2
import pytesseract
import numpy as np

def extract_text_from_image(uploaded_image):
    image = np.array(uploaded_image)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)[1]

    text = pytesseract.image_to_string(gray)
    return text
```

- Convert image to grayscale
- Adjust contrast
- Remove background noise
- Extract raw text

Output: Unstructured OCR text

Activity 2.3: AI Text Processing Module

File: ai_module.py

```python
import google.generativeai as genai

model = genai.GenerativeModel("models/gemini-2.5-flash")

def clean_and_structure_text(raw_text):
    prompt = f"""
    You are an AI note cleaner.
    1. Correct spelling and grammar.
    2. Reconstruct incomplete words.
    3. Format text properly.
    4. Extract action items as bullet points.

    Raw Text:
    {raw_text}
    """

    response = model.generate_content(prompt)
    return response.text
```

- Correct grammar **Input:**

```
i am go school tomorrow
```

Expected Output:

```
I am going to school tomorrow.
```

- Improve clarity
- Predict incomplete words
- Identify task-related phrases Output:
- Clean digital notes
- Separate to-do list

Activity 2.4: Stream lit UI Workflow

User Flow:

1. Upload handwritten image
2. Click “Convert”
3. OCR extracts text
4. Gemini refines content
5. Structured output displayed

Activity 2.5: File Export

Export Options:

- .txt for cleaned notes
- .csv for task list
- Future scope: PDF

```
st.download_button("Download Text", clean_text)
```

Milestone3: Flask Backend Integration

Objective:

Build a robust backend system with proper validation and modular design.

Core Backend Features

Input Validation

- Accepts JPG, PNG, JPEG
- Rejects empty uploads
- File size restriction
- Image readability verification

Error Handling	**OCR Handling Failure**
Gemini API	**Timeout Handling**
Structured JSON	**Error Responses**

Example Response:

```
{
  "status": "success",
  "clean_text": "...",
  "todo_list": [...]
}
```

Modular Architecture

```
scribble_to_digital/
│
├── app.py                # Main Flask server
├── routes/
│   └── process.py        # OCR + AI logic routes
├── services/
│   ├── ocr_service.py    # Image enhancement + OCR
│   ├── ai_service.py     # Gemini processing
│   └── export_service.py # PDF/Word export
├── utils/
│   └── validators.py     # File validation
└── uploads/
```

Export System

Supported Formats:

- PDF with formatted sections
- Word document (.docx)
- CSV task tracker
- File management Temporary Storage
- Automatic deletion

- Secure download handling

Milestone 4: Frontend Design (Streamlit UI)

Objective:

Design a modern, user-friendly interface.

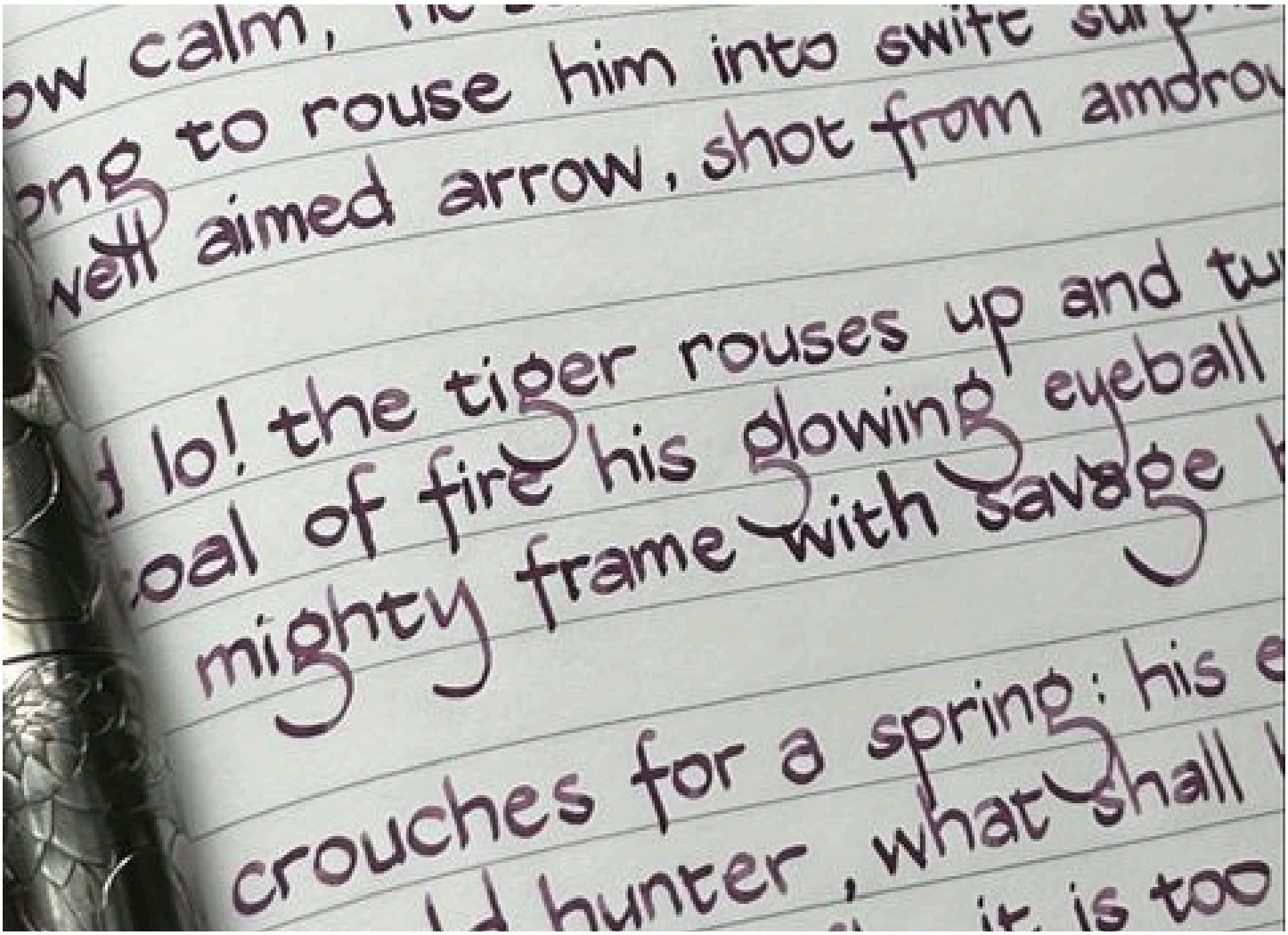
<u>Landing Page</u>

<u>Title:</u>

Digitize Your Handwritten Ideas with AI **<u>Features</u> :**

[Upload Notes] [Smart Correction] [To-Do Extraction]

- Upload Image



- AI-Powered Correction

✔Automatic Task Detection

Processing Page Components:

- Image uploader
- Convert button
- Reset option

```
-----------------------------------+---------
| ✍️ Scribble to Digital                     |
-----------------------------------------------
| Upload Handwritten Notes                   |
| [ Choose File ]                            |
-----------------------------------------------
| Image Preview                              |
-----------------------------------------------
| [ Convert to Digital ✨ ]                  |
-----------------------------------------------
| 📄 Clean Digital Text                      |
| [ Scrollable Text Area ]                   |
-----------------------------------------------
| ✅ Extracted To-Do List                    |
| [ Scrollable Text Area ]                   |
-----------------------------------------------
| [ Download as TXT ]  [ Download CSV ]      |
```

Export Section

Buttons:

- Download Clean Notes
- Download Task CSV User Feedback:
- Loading spinner
- Success message
- Error alerts

Milestone 5: Testing & Optimization

Testing Cases Case 1 – Clear Writing

Result: High accuracy extraction and clean output

Case 2 – Moderate Quality

Result: Minor OCR errors corrected by AI

Case 3 – Blurred Image

Result: Partial recovery using preprocessing

Case 4 – No Tasks Present

Result: Displays clean notes and “No action items found”

Quality Metrics

- OCR precision rate
- Grammar improvement score
- Task extraction accuracy
- Export formatting correctness

Optimization Techniques :

✔ Reduced API token usage

✔Improved image enhancement speed

✔Added loading indicators

✔Prevented duplicate processing ✔Handled API/network errors gracefully

Complete User Journey:

1. Open application
2. Upload handwritten note
3. Click Convert
4. View :
   - Clean formatted text
   - Extracted tasks
5. Download results

Conclusion :

The *Scribble to Digital* application successfully demonstrates how Generative AI and OCR technology can work together to transform handwritten notes into structured digital content. By integrating image preprocessing, Tesseract OCR, and Gemini 2.5 Flash, the system effectively converts messy handwritten text into grammatically correct and well-formatted digital notes.

The application not only enhances readability but also intelligently extracts actionable tasks, making it highly useful for students, professionals, and daily planners. The modular architecture using Streamlit and Flask ensures scalability, maintainability, and smooth performance.

Through comprehensive testing and optimization, the system proved to be stable, accurate, and user-friendly even under different image qualities and edge cases.

Overall, Scribble to Digital bridges the gap between traditional handwritten notes and modern digital productivity tools, showcasing the practical impact of Generative AI in real-world applications.